

选择题

1. (B) 在软件开发的过程中，若能推迟暴露其中的错误，则为修复和改正错误所花费的代价就会降低。

A. 真
B. 假

分析：软件工程的一个核心原则是：缺陷发现得越晚，修复成本越高。在需求阶段发现错误的修复成本可能只是编码阶段的几分之一，而到了生产环境（维护阶段），成本将呈指数级增长。

2. (A) 好的测试是用少量测试用例运行程序，发现被测程序尽可能多的错误。

A. 真
B. 假

分析：测试的目的是以最小的代价（最少的用例）发现最多的缺陷。

3. (B) 好的测试用例应能证明软件是正确的。

A. 真
B. 假

分析：程序测试只能证明错误的存在，而不能证明错误不存在。测试永远无法穷举所有路径，因此无法证明软件完全正确，只能通过发现bug来提高质量信心。

4. (B) 白盒测试仅与程序的内部结构有关，完全可以不考虑程序的功能要求。

A. 真
B. 假

分析：虽然白盒测试主要关注内部逻辑结构（控制流、数据流），但测试的最终目的仍然是验证软件是否符合需求。如果内部逻辑正确但实现了错误的功能（即不符合功能规格说明），测试也是失败的。白盒测试是基于“内部逻辑是否正确实现了功能”来开展的。

5. (A) 等价类划分方法将所有可能的输入数据划分成若干部分，然后从每一部分中选取少数有代表性的数据作为测试用例。

A. 真
B. 假

分析：等价类划分的核心假设是：如果等价类中的一个数据能发现（或不发现）错误，那么该类中的其他数据也能发现（或不发现）错误。

6. 使用独立测试团队的最好理由是 (C)。

A. 软件开发人员不需要做任何测试
B. 测试人员在测试开始之前不参与项目
C. 测试团队将更彻底地测试软件
D. 开发人员与测试人员之间的争论会减少

分析：开发人员往往难以发现自己代码中的错误（“开发者思维”的偏见）。独立测试团队能提供更客观、更彻底的视角。

7. 类的行为应该基于（ D ）进行测试。

- A. 数据流图
- B. 用例图
- C. 对象图
- D. 状态图

分析：类的行为通常与对象的状态变化有关。状态图描述了对象生命周期中状态的转换，是测试对象行为（特别是状态驱动的行为）的最佳依据。

8. 下面的（ ABDE ）说法是正确的。

- A. 恢复测试是以各种方式迫使软件失效从而检测软件是否能够继续执行的一种系统测试。
- B. 安全测试是检测系统中的保护机制是否可以保护系统免受非正常的攻击。
- C. 压力测试是检测在极限环境中使用系统时施加在用户上的压力。
- D. 功能测试是根据软件需求规格说明和测试需求列表，验证产品的功能实现是否符合需求规格。
- E. 安装测试是保证应用程序能够被成功地安装。

分析：ABDE说法都是正确的。C——压力测试是指在非正常的、超负荷的条件下（如资源不足、高并发）测试系统的稳定性，而不是测试用户的压力。

作业

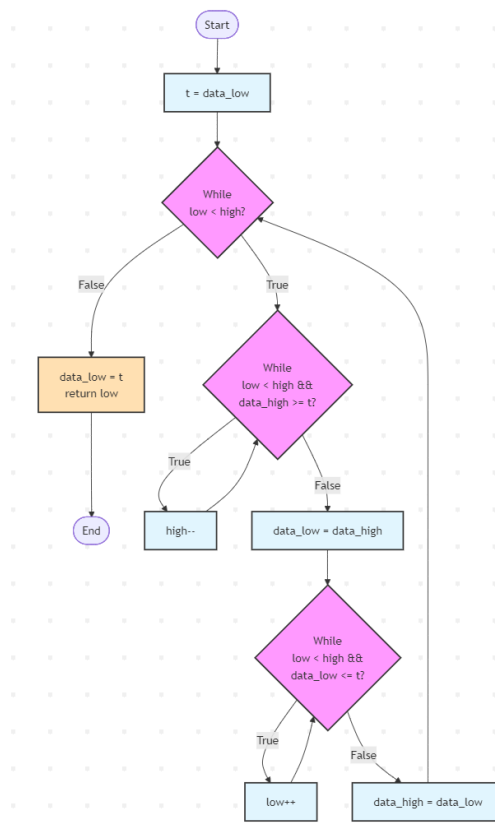
1. 理解下面的程序结构，请使用基本路径方法设计该程序测试用例。

```
#include <stdio.h>
int partition(int *data, int low, int high)
{
    int t = 0;

    t = data[low];
    while (low < high) {
        while (low < high && data[high] >= t)
            high--;
        data[low] = data[high];
        while (low < high && data[low] <= t)
            low++;
        data[high] = data[low];
    }
    data[low] = t;

    return low;
}
```

(1) 程序逻辑与控制流分析 (CFG 构建)



(2) 计算环路复杂度: $V(G)=\text{判定节点数量}+1=4$ 。这意味着我们需要设计 4 条独立的测试路径。

(3) 确定独立路径

我们需要找出 4 条逻辑上独立的路径，覆盖所有的判定结果（真/假）：

- 路径 1 (不进入外层循环):
 - 条件: `low >= high` 初始即成立。
 - 路径序列: 1 (False) -> 8
- 路径 2 (进入外层，执行内层循环1，跳过内层循环2主体):
 - 这在逻辑上比较难完全构造，因为如果跳过内层循环1的Swap，数据通常满足进入内层循环2的条件。我们要找的是主要覆盖内层循环1的路径。
 - 关键特征: `data[high] >= t` 成立至少一次。
- 路径 3 (进入外层，跳过内层循环1主体，执行内层循环2):
 - 关键特征: `data[high] < t` (内层1直接失败)，发生交换，然后 `data[low] <= t` (内层2成立)。
- 路径 4 (混合/典型路径):
 - 覆盖循环回跳或多次迭代的情况，确保所有分支都被执行过。

(4) 设计测试用例

用例编号	覆盖路径逻辑	输入数据 (data数组, low, high)	预期执行流程 / 结果
TC-01	路径 1 不满足外层循环条件	data: [10] low: 0 high: 0	程序初始化 t，判断 <code>0 < 0</code> 为假，直接跳转到最后。 结果: 返回 0，数组不变。
TC-02	路径 2 触发内层循环 1 (high--)	data: [5, 8] low: 0 high: 1	1. t=5。 2. 外层 <code>0 < 1</code> (真)。 3. 内层 <code>1 * 8 >= 5</code> (真) -> high 变为 0。 4. 内层 <code>1 * 0 < 0</code> (假) -> 退出。 5. 交换(自身)。内层 <code>2 * 0 < 0</code> (假)。 6. 外层 <code>0 < 0</code> (假)。 结果: 返回 0，数组 [5, 8] (5已归位)。
TC-03	路径 3 触发内层循环 2 (low++)	data: [5, 2] low: 0 high: 1	1. t=5。 2. 外层 <code>0 < 1</code> (真)。 3. 内层 <code>1 * 2 >= 5</code> (假)。 4. data[0] = 2。 5. 内层 <code>2 * 2 <= 5</code> (真) -> low 变为 1。 6. 内层 <code>2 * 1 < 1</code> (假)。 7. data[1] = data[1]。 8. 外层 <code>1 < 1</code> (假)。 结果: 返回 1，数组 [2, 5]。
TC-04	路径 4 完整路径 (混合)	data: [5, 8, 2] low: 0 high: 2	1. t=5。 2. 内层1: <code>2 >= 5</code> (假)。 3. Swap: data[0]=2。 4. 内层2: <code>2 <= 5</code> (真, low=1); <code>8 <= 5</code> (假)。 5. Swap: data[2]=8。 6. 此时 data 为 [2, 8, 8], low=1, high=2。继续外层循环... 结果: 返回 1，数组变为 [2, 5, 8]。

2. 请使用等价类划分和边界值分析的方法，给出 `getNumDaysInMonth(int month, int year)` 方法的测试用例，其中 `getNumDaysInMonth` 方法根据给定的月份和年份返回该月份的总天数。

(1) 请给出划分的等价类。

输入参数	有效等价类(Valid)	无效等价类(Invalid)
Month (月份)	M1: 31天的月份 {1, 3, 5, 7, 8, 10, 12}	M4: Month < 1 (如 0, -1)
	M2: 30天的月份 {4, 6, 9, 11}	M5: Month > 12 (如 13, 100)
	M3: 特殊月份 {2}	
Year (年份)	Y1: 闰年 (能被400整除，或能被4整除但不能被100整除)	Y3: Year < 1 (假设年份必须为正整数)
	Y2: 平年 (不能被4整除，或能被100整除但不能被400整除)	

(2) 测试用例设计:

序号	输入参数(Month,Year)	覆盖的等价类/边界条件	期望输出
1	month=1, year=2023	M1 (31天月边界), Y2 (普通平年)	31
2	month=4, year=2023	M2 (30天月), Y2	30
3	month=2, year=2024	M3 (2月), Y1 (普通闰年: 被4整除)	29
4	month=2, year=2000	M3 (2月), Y1 (世纪闰年: 被400整除)	29
5	month=2, year=2023	M3 (2月), Y2 (普通平年)	28
6	month=2, year=1900	M3 (2月), Y2 (世纪平年: 被100整除但非400)	28
7	month=12, year=2023	M1 (31天月边界), Y2	31
8	month=0, year=2023	M4 (无效月份下边界)	Error / 0
9	month=13, year=2023	M5 (无效月份上边界)	Error / 0
10	month=8, year=2023	M1 (31天月非边界), Y2	31

3. 现在要对一个咨询公司的薪酬支付软件进行功能测试，该软件的规格说明如下：

对于每周工作超过 40 小时的咨询人员，前 40 小时按照他们的小时薪酬计算，多余的小时数按照双倍小时薪酬计算；对于每周工作不足 40 小时的咨询人员，按照他们的小时薪酬计算，并生成一份缺勤报告；对于每周工作超过 40 小时的长期工作人员，直接按照他们的固定工资计算。

(1) 使用决策表（格式如下）描述上述的薪酬支付规则；
考虑咨询人员的三种时间状态 (>40, <40, =40) 以及长期人员的状态。

规则		规则1	规则2	规则3	规则4
条件桩	是否是咨询人员	Y	Y	N	N
	每周工作 >= 40小时	Y	N	Y	N
动作桩	薪酬 = 计算: $40 \times \text{时薪} + (\text{总工时} - 40) \times 2 \times \text{时薪}$	✓			未定义
	薪酬 = 总小时数 $\times$ 时薪		✓		
	薪酬 = 生成缺勤报告		✓		
	薪酬 = 按固定工资计算			✓	

(2) 根据上述决策表，使用下表列出自己设计的测试用例。

序号	输入参数 (身份, 周工作时间)	期望输出
1	咨询人员, 45小时	薪酬 = $40 \times \text{时薪} + 5 \times 2 \times \text{时薪}$
2	咨询人员, 30小时	薪酬 = $30 \times \text{时薪}$ ，并生成缺勤报告
3	咨询人员, 40小时	薪酬 = $40 \times \text{时薪}$
4	长期工作人员, 45小时	薪酬 = 固定工资
5	长期工作人员, 40小时	薪酬 = 固定工资
6	长期工作人员, 35小时	未定义

4. 下面给出的是购买车票（PurchaseTicket）用例的正常流，除此之外还应包括无零钱找（NoChange）、缺票（OutofOrder）、超时（TimeOut）和取消（Cancel）等四个备选流，请结合场景法和其他方法设计测试用例。

用例名称	购买车票
前置条件	乘客站在售票机前，有足够的钱买车票。
正常流	1. 乘客选择需要达到的地点，如果按下了多个地点按钮，售票机只考虑最后一次按下的地点。 2. 售票机显示出应付款数。 3. 乘客投入钱。 4. 如果乘客在投入足够的钱之前选择了新的地点，售票机应该把所有的钱退还乘客。 5. 如果乘客投入的钱比应付款多，售票机应该退出多余的零钱。 6. 售票机给出车票。 7. 乘客拿走找零和车票。
后置条件	乘客买到了他选择的车票。

要求：使用下表列出自己设计的测试用例。

我们需要覆盖正常流以及无零钱（NoChange）、缺票（OutofOrder）、超时（TimeOut）和取消（Cancel）这四个备选流。此外，第1点（多次按键）和第4点（中途换目的地）也是重要的业务逻辑场景。因此，设计以下7个测试用例：

测试用例编号	TC_001
测试标题	正常购买车票（需找零）
预置条件	售票机正常运行，钱箱中有足够的零钱，票纸充足。
操作步骤	1. 乘客按下“地点A”按钮（票价5元）。2. 售票机显示应付5元。3. 乘客投入10元纸币。
预期输出	1. 售票机出票口吐出一张去往“地点A”的车票。2. 找零口吐出5元找零。3. 屏幕恢复初始状态。

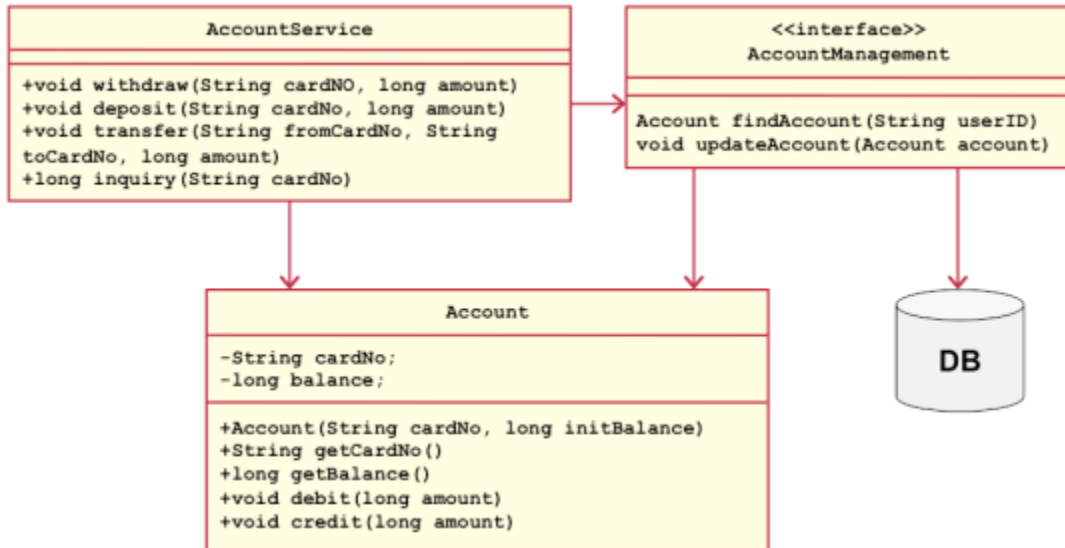
测试用例编号	TC_002
测试标题	无零钱找零（NoChange）
预置条件	售票机内的找零钱箱为空（或不足）。
操作步骤	1. 乘客按下“地点A”按钮（票价5元）。2. 乘客投入10元纸币（需要找零的情况）。
预期输出	1. 售票机提示“本机无零钱”或“请投入精确金额”。2. 售票机退还乘客投入的10元纸币。3. 不出票。

测试用例编号	TC_003
测试标题	缺票无法购买（OutofOrder）
预置条件	售票机内部打印纸已用完。
操作步骤	1. 乘客尝试按下“地点A”按钮。

预期输出	1. 售票机屏幕显示“暂停服务”或“缺票”提示。 2. 投币口关闭或拒绝接收纸币。 3. 无法进行后续购买操作。
测试用例编号	TC_004
测试标题	操作超时（TimeOut）
预置条件	售票机正常。
操作步骤	1. 乘客按下“地点A”按钮。 2. 乘客投入部分金额（例如2元，不足票价）。 3. 乘客停止操作，等待超过设定时间（如60秒）。
预期输出	1. 售票机提示“操作超时”。 2. 售票机自动退还已投入的2元。 3. 交易取消，屏幕返回初始欢迎界面。
测试用例编号	TC_005
测试标题	用户取消操作（Cancel）
预置条件	售票机正常。
操作步骤	1. 乘客按下“地点A”按钮。 2. 乘客投入部分金额。 3. 乘客按下“取消”按钮。
预期输出	1. 售票机立即退还已投入的所有金额。 2. 交易终止，不出票。 3. 屏幕返回初始欢迎界面。
测试用例编号	TC_006
测试标题	投币前多次选择地点
预置条件	售票机正常。
操作步骤	1. 乘客先按下“地点A”按钮（票价5元）。 2. 乘客紧接着按下“地点B”按钮（票价10元）。 3. 乘客投入10元。
预期输出	1. 步骤2后，屏幕应显示“地点B”的票价（10元）。 2. 投入10元后，售票机打印一张去往地点B的车票。 3. 无找零。
测试用例编号	TC_007
测试标题	投币中途更改目的地
预置条件	售票机正常。
操作步骤	1. 乘客按下“地点A”按钮（票价5元）。 2. 乘客投入2元（不足票价）。 3. 乘客在补款前，按下了“地点B”按钮。
预期输出	1. 根据需求描述，售票机立即退还已投入的2元。 2. 系统更新为购买“地点B”的状态（显示地点B的应付款）。

作业

下面的 UML 图显示了银行卡账户的服务功能：



要求：

- (1) 请使用Java 或 C++语言和测试驱动开发方法编写上述程序，并进行单元测试和代码覆盖分析；
- 架构：遵循UML图，实现了 **Account (实体)**, **AccountService (业务逻辑)**, 和 **AccountManagement (接口/抽象类)**。
 - 数据模拟：编写了 `MockAccountDatabase` 类来实现 `AccountManagement` 接口，使用 `std::map` 在内存中存储数据。
 - 测试：使用 `assert` 宏和 `try-catch` 块进行单元测试，**设计了16个测试，覆盖了正常业务流程（存款、取款、转账）、边界条件测试（取光余额、零余额账户操作）、输入合法性测试（存取款负数、存取0元等）和异常流程（余额不足、账户不存在）**

C++代码如下：

```
#include <iostream>
#include <string>
#include <vector>
#include <map>
#include <stdexcept>
#include <cassert>
#include <cstdlib>

using namespace std;

// =====
// 1. 领域实体：Account
```

```

// =====
class Account {
private:
    string cardNo;
    long balance;

public:
    Account(string cardNo, long initBalance) : cardNo(cardNo),
    balance(initBalance) {
        if (initBalance < 0) throw invalid_argument("初始化余额不能为负");
    }

    string getCardNo() const { return cardNo; }

    long getBalance() const { return balance; }

    // 存款
    void credit(long amount) {
        if (amount <= 0) throw invalid_argument("存款金额必须大于0");
        balance += amount;
    }

    // 取款
    void debit(long amount) {
        if (amount <= 0) throw invalid_argument("取款金额必须大于0");
        if (balance < amount) throw runtime_error("余额不足");
        balance -= amount;
    }
};

// =====
// 2. 接口: AccountManagement
// =====
class AccountManagement {
public:
    virtual Account* findAccount(string userID) = 0;
    virtual void updateAccount(Account* account) = 0;
    virtual ~AccountManagement() {}
};

// =====
// 3. 基础设施: 模拟数据库 (Mock Database)
// =====
class MockAccountDatabase : public AccountManagement {
private:
    map<string, Account*> db;

public:

```

```

void insertAccount(Account* account) {
    if (db.find(account->getCardNo()) != db.end()) {
        delete db[account->getCardNo()];
    }
    db[account->getCardNo()] = account;
}

Account* findAccount(string cardNo) {
    if (db.find(cardNo) != db.end()) {
        return db[cardNo];
    }
    return NULL; // 使用 NULL 替代 nullptr
}

void updateAccount(Account* account) {
    if (account && db.find(account->getCardNo()) != db.end()) {
        db[account->getCardNo()] = account;
    }
}

// 清空内存
void clear() {
    for (map<string, Account*>::iterator it = db.begin(); it !=
db.end(); ++it) {
        delete it->second;
    }
    db.clear();
}

~MockAccountDatabase() {
    clear();
}

};

// =====
// 4. 服务层: AccountService
// =====
class AccountService {
private:
    AccountManagement* accountManager;

public:
    AccountService(AccountManagement* am) : accountManager(am) {}

    void deposit(string cardNo, long amount) {
        Account* acc = accountManager->findAccount(cardNo);
        if (!acc) throw runtime_error("账户不存在: " + cardNo);
    }
}

```

```

        acc->credit(amount);
        accountManager->updateAccount(acc);
    }

    void withdraw(string cardNo, long amount) {
        Account* acc = accountManager->findAccount(cardNo);
        if (!acc) throw runtime_error("账户不存在: " + cardNo);

        acc->debit(amount);
        accountManager->updateAccount(acc);
    }

    long inquiry(string cardNo) {
        Account* acc = accountManager->findAccount(cardNo);
        if (!acc) throw runtime_error("账户不存在: " + cardNo);

        return acc->getBalance();
    }

    void transfer(string fromCardNo, string toCardNo, long amount) {
        if (fromCardNo == toCardNo) throw invalid_argument("不能向自己转账");

        Account* src = accountManager->findAccount(fromCardNo);
        Account* dest = accountManager->findAccount(toCardNo);

        if (!src) throw runtime_error("转出账户不存在");
        if (!dest) throw runtime_error("转入账户不存在");

        // 事务模拟
        src->debit(amount);
        dest->credit(amount);

        accountManager->updateAccount(src);
        accountManager->updateAccount(dest);
    }
};

// =====
// 5. 测试套件
// =====

#define TEST_EXCEPTION(statement, exceptionType, expectedMsg, testName) \
    try { \
        statement; \
        cout << "[FAIL] " << testName << " (未抛出预期异常)" << endl; \
        assert(false); \
    } catch (const exceptionType& e) { \
        string msg = e.what(); \

```

```

        if (msg.find(expectedMsg) != string::npos) { \
            cout << "[PASS] " << testName << endl; \
        } else { \
            cout << "[FAIL] " << testName << " (异常消息不匹配: " << msg <<
)" << endl; \
            assert(false); \
        } \
    } catch (...) { \
        cout << "[FAIL] " << testName << " (抛出了错误的异常类型)" << endl; \
        assert(false); \
    }
}

// 辅助函数: 重置数据
void resetData(MockAccountDatabase* db) {
    db->clear();
    db->insertAccount(new Account("A1", 1000));
    db->insertAccount(new Account("A2", 500));
    db->insertAccount(new Account("ZERO", 0));
}

void runTests() {
    cout << "==== 单元测试 =====" << endl;

    MockAccountDatabase* db = new MockAccountDatabase();
    AccountService service(db);

    // --- 组 1: 正常业务流程 ---
    resetData(db);
    cout << "\n--- Group 1: 正常业务流程 ---" << endl;

    // Case 1: 正常存款
    service.deposit("A1", 100);
    assert(service.inquiry("A1") == 1100);
    cout << "[PASS] 正常存款" << endl;

    // Case 2: 正常取款
    service.withdraw("A1", 100);
    assert(service.inquiry("A1") == 1000);
    cout << "[PASS] 正常取款" << endl;

    // Case 3: 正常转账
    service.transfer("A1", "A2", 200);
    assert(service.inquiry("A1") == 800);
    assert(service.inquiry("A2") == 700);
    cout << "[PASS] 正常转账" << endl;

    // --- 组 2: 边界条件测试 ---
    resetData(db);

```

```

cout << "\n--- Group 2: 边界条件测试 ---" << endl;

// Case 4: 取光余额
service.withdraw("A2", 500);
assert(service.inquiry("A2") == 0);
cout << "[PASS] 边界取款 (取光)" << endl;

// Case 5: 零余额账户操作
service.deposit("ZERO", 50);
assert(service.inquiry("ZERO") == 50);
cout << "[PASS] 零余额账户操作" << endl;

// --- 组 3: 输入合法性测试 (Negative/Zero) ---
resetData(db);
cout << "\n--- Group 3: 输入合法性测试 ---" << endl;

// Case 6: 存款负数
TEST_EXCEPTION(service.deposit("A1", -100), invalid_argument, "存款金额
必须大于 0", "存款负数拦截");

// Case 7: 存款 0 元
TEST_EXCEPTION(service.deposit("A1", 0), invalid_argument, "存款金额必须
大于 0", "存款 0 元拦截");

// Case 8: 取款负数
TEST_EXCEPTION(service.withdraw("A1", -50), invalid_argument, "取款金额
必须大于 0", "取款负数拦截");

// Case 9: 初始化余额为负 (构造函数)
// 注意: 在宏中使用对象声明需要小心, 这里为了简单单独写 try-catch 或者利用临
时对象
try {
    Account bad("BAD", -100);
    assert(false);
} catch (const invalid_argument& e) {
    cout << "[PASS] 账户初始化负余额拦截" << endl;
}

// --- 组 4: 业务逻辑异常测试 ---
resetData(db);
cout << "\n--- Group 4: 业务逻辑异常测试 ---" << endl;

// Case 10: 余额不足取款
TEST_EXCEPTION(service.withdraw("A2", 501), runtime_error, "余额不足", "
余额不足取款");

// Case 11: 余额不足转账

```

```

    TEST_EXCEPTION(service.transfer("A2", "A1", 600), runtime_error, "余额不足", "余额不足转账");

    // 验证原子性
    assert(service.inquiry("A2") == 500);
    assert(service.inquiry("A1") == 1000);
    cout << "[PASS] 转账事务原子性验证" << endl;

    // --- 组 5: 账户存在性与关联测试 ---
    resetData(db);
    cout << "\n--- Group 5: 账户存在性与关联测试 ---" << endl;

    // Case 12: 查询不存在的账户
    TEST_EXCEPTION(service.inquiry("GHOST"), runtime_error, "账户不存在", "查询不存在账户");

    // Case 13: 转出账户不存在
    TEST_EXCEPTION(service.transfer("GHOST", "A1", 100), runtime_error, "转出账户不存在", "转出账户不存在");

    // Case 14: 转入账户不存在
    TEST_EXCEPTION(service.transfer("A1", "GHOST", 100), runtime_error, "转入账户不存在", "转入账户不存在");

    // Case 15: 自我转账
    TEST_EXCEPTION(service.transfer("A1", "A1", 100), invalid_argument, "不能向自己转账", "自我转账拦截");

    // Case 16: 复杂操作链
    service.deposit("A1", 100); // 1100
    service.withdraw("A1", 50); // 1050
    service.transfer("A1", "A2", 50); // A1: 1000, A2: 550
    assert(service.inquiry("A1") == 1000);
    cout << "[PASS] 复杂操作链后状态一致性" << endl;

    delete db;
    cout << "\n===== 所有 16 个测试用例通过 =====" << endl;
}

int main() {
    runTests();
    return 0;
}

```

运行代码，设计的16个测试用例全部通过：

```

===== 单元测试 =====

--- Group 1: 正常业务流程 ---
[PASS] 正常存款
[PASS] 正常取款
[PASS] 正常转账

--- Group 2: 边界条件测试 ---
[PASS] 边界取款（取光）
[PASS] 零余额账户操作

--- Group 3: 输入合法性测试 ---
[PASS] 存款负数拦截
[PASS] 存款0元拦截
[PASS] 取款负数拦截
[PASS] 账户初始化负余额拦截

--- Group 4: 业务逻辑异常测试 ---
[PASS] 余额不足取款
[PASS] 余额不足转账
[PASS] 转账事务原子性验证

--- Group 5: 账户存在性与关联测试 ---
[PASS] 查询不存在账户
[PASS] 转出账户不存在
[PASS] 转入账户不存在
[PASS] 自我转账拦截
[PASS] 复杂操作链后状态一致性

===== 所有 16 个测试用例通过 =====

```

代码覆盖分析：使用MinGW GCC + Gcov，采用了白盒测试方法，针对“转账限额3000元”的新需求及基础存取款功能进行了覆盖测试。

```

(base) PS D:\desktop> .\gcov 未命名1.cpp
File 'D:/desktop/未命名1.cpp'
Lines executed:98.47% of 131
Creating '未命名1.cpp.gcov'

```

代码逻辑覆盖率98.47%，核心业务逻辑（存款、取款、转账、限额判断、余额判断）均已被执行。根据.gcov文件显示，未覆盖的2行代码都是未触发的防御性代码分支。

```

-: 232:     try {
2: 233:         Account bad("BAD", -100);
#####: 234:         assert(false);
1: 235:     } catch (const invalid_argument& e) {
1: 236:         cout << "[PASS] 账户初始化负余额拦截" << endl;
-: 237:     }
-: 238: }

15: 62:     if (db.find(account->getCardNo()) != db.end()) {
#####: 63:         delete db[account->getCardNo()];
-: 64:     }

```

(2) 如果银行卡的转账服务增加一个新的需求“一次转账金额限定为 3000 元”，请修改代码并重新进行单元测试和覆盖分析。

增加新需求，一次转账金额不超过 3000 元。需要在服务层 (AccountService) 的转账逻辑添加业务规则校验。

```
void transfer(string fromCardNo, string toCardNo, long amount) {
    if (fromCardNo == toCardNo) throw invalid_argument("不能向自己转账");

    if (amount > TRANSFER_LIMIT) {
        throw runtime_error("转账金额超过单笔限额 (Max: 3000)");
    }

    Account* src = accountManager->findAccount(fromCardNo);
    Account* dest = accountManager->findAccount(toCardNo);

    if (!src) throw runtime_error("转出账户不存在");
    if (!dest) throw runtime_error("转入账户不存在");

    src->debit(amount);
    dest->credit(amount);

    accountManager->updateAccount(src);
    accountManager->updateAccount(dest);
}
```

同时，在测试设计需要新增针对限额的边界值测试。

```
// Case 1: 3000 (成功)
try {
    service.transfer("RICH", "A1", 3000);
    assert(service.inquiry("RICH") == 7000);
    cout << "[PASS] 限额边界测试 (转账 3000 成功)" << endl;
} catch (...) {
    assert(false);
}

// Case 2: 3001 (失败)
TEST_EXCEPTION(service.transfer("RICH", "A1", 3001), runtime_error, "超过单笔限额", "限额拦截测试 (3001)");

// Case 3: 余额不足优先于限额
TEST_EXCEPTION(service.transfer("A2", "A1", 600), runtime_error, "余额不足", "余额不足检查");
```

重新运行修改后的代码，转账限额测试组也全部pass:

```
--- Group: 转账限额测试 ---
[PASS] 限额边界测试 (转账 3000 成功)
[PASS] 限额拦截测试 (3001)
[PASS] 余额不足检查
```

代码覆盖分析：使用MinGW GCC + Gcov，采用了白盒测试方法，针对“转账限额3000元”的新需求及基础存取款功能进行了覆盖测试。

```
(base) PS D:\desktop> .\gcov 未命名1.cpp
File 'D:/desktop/未命名1.cpp'
Lines executed:96.08% of 102
Creating '未命名1.cpp.gcov'
```

代码逻辑覆盖率96.08%，核心业务逻辑（存款、取款、转账、限额判断、余额判断）均已被执行。根据.gcov文件显示，未覆盖的3行代码都是未触发的防御性代码分支。

```

-: 232:    try {
2: 233:        Account bad("BAD", -100);
#####: 234:        assert(false);
1: 235:    } catch (const invalid_argument& e) {
1: 236:        cout << "[PASS] 账户初始化负余额拦截" << endl;
-: 237:    }

15: 62:    if (db.find(account->getCardNo()) != db.end()) {
#####: 63:        delete db[account->getCardNo()];
-: 64:    }

13: 67:    Account* findAccount(string cardNo) {
13: 68:        if (db.find(cardNo) != db.end()) {
13: 69:            return db[cardNo];
-: 70:        }
#####: 71:        return NULL; // C++98 use NULL
-: 72:    }
```